

DSA STUDY BLUEPRINT SERIES

40. Sudoku Solver Backtracking

Solving Sudoku Grids using Backtracking

Section 06 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Constraint Search:** Solve sudoku cells by validating digits 1-9.
- Revert changes if configuration leads to an invalid state.

Memory & Execution Blueprint

Grid Cell check: Row OK • Col OK • Sub-grid 3x3 OK ⇒ Place digit

C++ Implementation Reference

Standard code layout and syntax

```
bool solve(vector>& board) {  
    for (int r=0; r<9; r++) {  
        for (int c=0; c<9; c++) {  
            if (board[r][c] == '.') {  
                for (char val='1'; val<='9'; val++) {  
                    if (isSafe(board, r, c, val)) {  
                        board[r][c] = val;  
                        if (solve(board)) return true;  
                        board[r][c] = '.'; // Backtrack  
                    }  
                }  
            }  
            return false;  
        }  
    }  
    return true;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(9^{81})$ worst case
Space Complexity	$O(81)$ stack

Key Practices & Gotchas

- **Important:** Check safety by verifying row, col, and the local 3x3 sub-grid.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace backtracking cell navigations in a grid puzzle:

Cell visited	Direction Checked	Move Validation	Dead-end Backtrack step
(0, 0)	Down (D) → (1, 0)	Valid (Open path)	Proceed to cell (1, 0)
(1, 0)	Right (R) → (1, 1)	Invalid (Blocked wall)	Revert to cell (1, 0) and try Next check

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pruning Searches:** Check safety criteria (e.g. diagonal checks for N-Queens) to skip bad board configurations immediately.
- **Visited state maps:** Mark board grid tiles as visited inline to avoid loops, resetting them when returning.
- **Related LeetCode Problems:** #51 N-Queens, #37 Sudoku Solver, #79 Word Search.